

计算机图形学实验报告

实验名称	实验一 直线的生成		
评分		实验日期	2026 年 5 月 9 日
指导教师	霍鑫	姓名	张致远
专业班级	信计 23-1	学号	2023213980

1 实验目的

1. 理解光栅图形系统中直线段扫描转换的基本思想，体会“用离散像素逼近连续直线”这一图形学核心问题。
2. 掌握 DDA 算法、中点画线算法以及 Bresenham 算法的数学推导与实现细节。
3. 通过对三种算法的实现与比较，分析它们在效率、误差、可移植性等方面的差异，重点掌握 **Bresenham 直线生成算法** 的递推逻辑。
4. 熟悉 OpenGL 编程基本框架，能够利用 `GL_POINTS` 模式按算法逐像素绘制图元。

2 实验要求

实验环境：

- 实验设备：PC 机一台
- 操作系统：Windows 11
- 开发工具：Visual Studio 2022 + OpenGL (freeglut 3.4.0)
- 编程语言：C++

实验内容：

1. 在屏幕上分别用 DDA 算法、中点画线算法、Bresenham 算法绘制同一条直线段；
2. 不同算法采用不同颜色或线型加以区别，便于对比生成结果；
3. 通过键盘交互切换显示某一种或全部算法的输出，便于观察像素分布；
4. 重点验证 Bresenham 算法在 $0 \leq k \leq 1$ 第一象限的正确性，并扩展到任意斜率。

3 关键算法及实现原理

设直线段两端点为 $P_1(x_1, y_1)$ 与 $P_2(x_2, y_2)$, 记 $\Delta x = x_2 - x_1$, $\Delta y = y_2 - y_1$, 斜率 $k = \Delta y / \Delta x$ 。不失一般性, 下文以 $0 \leq k \leq 1$ 、 $x_1 < x_2$ 为典型情况推导。

3.1 DDA (数字微分分析器) 算法

DDA 利用直线的微分方程 $\frac{dy}{dx} = k$, 沿 x 方向每步增 1, 则 y 方向相应增加 k :

$$x_{i+1} = x_i + 1, \quad y_{i+1} = y_i + k.$$

绘制时取 $\text{round}(y_{i+1})$ 作为像素纵坐标。算法直观, 但需要浮点加法和取整运算, 效率较低。

3.2 中点画线算法

将直线方程写为 $F(x, y) = ax + by + c = 0$, 其中 $a = \Delta y$, $b = -\Delta x$ 。当像素 (x_i, y_i) 已绘制后, 下一像素只可能是 $E(x_i + 1, y_i)$ 或 $NE(x_i + 1, y_i + 1)$ 。取两者中点 $M(x_i + 1, y_i + \frac{1}{2})$, 构造判别式

$$d_i = F(x_i + 1, y_i + \frac{1}{2}) = a(x_i + 1) + b(y_i + \frac{1}{2}) + c.$$

$d_i < 0$ 时中点在直线下方, 下一像素取 NE ; 否则取 E 。利用增量推导得:

$$d_{i+1} = \begin{cases} d_i + a + b, & d_i < 0 \text{ (走 } NE) \\ d_i + a, & d_i \geq 0 \text{ (走 } E) \end{cases}$$

初值 $d_1 = a + b/2$ 。为消除小数, 可整体乘 2, 即令 $D = 2d$, 全部用整数运算。

3.3 Bresenham 算法 (重点)

Bresenham 同样采用递推思想, 但以“误差量”代替“中点判别式”, 物理含义更直观。设当前像素为 (x_i, y_i) , 理想直线在 $x_i + 1$ 处的纵坐标为 $y_i + k$, 当累计偏离 ≥ 0.5 时纵坐标进 1。

为避免浮点运算, 令

$$P_i = 2\Delta x \cdot e_i - \Delta x,$$

则判别条件 $e_i \geq 0.5$ 等价于 $P_i \geq 0$ 。代入递推得:

$$P_{i+1} = \begin{cases} P_i + 2\Delta y - 2\Delta x, & P_i \geq 0 \\ P_i + 2\Delta y, & P_i < 0 \end{cases} \quad P_1 = 2\Delta y - \Delta x.$$

算法步骤 ($0 \leq k \leq 1$):

1. 画起点 (x_1, y_1) ，计算误差初值 $P_1 = 2\Delta y - \Delta x$ ，置 $i = 1$ ；
2. 求下一像素 $x_{i+1} = x_i + 1$ ；若 $P_i \geq 0$ 则 $y_{i+1} = y_i + 1$ ，否则 $y_{i+1} = y_i$ ；
3. 画点 (x_{i+1}, y_{i+1}) ；
4. 更新误差：若 $P_i \geq 0$ 时 $P_{i+1} = P_i + 2\Delta y - 2\Delta x$ ，否则 $P_{i+1} = P_i + 2\Delta y$ ；
5. $i = i + 1$ ，若 $i < \Delta x + 1$ 转步骤 2，否则结束。

对一般斜率的推广：若 $|k| > 1$ ，则交换 x, y 角色（沿 y 步进）；若 $\Delta x < 0$ 或 $\Delta y < 0$ ，通过 sign 控制步进方向即可，本质算法保持不变。

3.4 三种算法对比

对比维度	DDA	中点画线	Bresenham
基本运算	浮点加 + 取整	整数加	整数加（仅加减）
单像素开销	较大	小	最小
误差分析	浮点累计误差	几何意义清晰	误差离散可控
推广性	易扩展到曲线	易扩展到圆/椭圆	易扩展到圆/椭圆
工程应用	教学 / 原型	较常用	硬件实现首选

表 1: 三种直线生成算法的比较

4 程序调试中的问题

1. **斜率范围的限制问题。**最初仅按指导书给定步骤实现 $0 \leq k \leq 1$ 的情况，当输入端点斜率为负或大于 1 时输出明显错误。
解决：在算法入口对端点进行预处理——根据 $|\Delta x|$ 与 $|\Delta y|$ 的大小决定主步进方向，并通过 $\text{sign}(\Delta x)$ 、 $\text{sign}(\Delta y)$ 控制步进方向，使 Bresenham 适用于全部 8 个八分圆。
2. **OpenGL 坐标系与窗口像素的映射问题。**默认正交投影中，`glVertex2i` 接收的是世界坐标，与窗口像素并非一一对应。若不显式设置投影矩阵，绘制结果会被自动缩放，难以观察单像素差异。
解决：调用 `gluOrtho2D(0, W, 0, H)` 将世界坐标显式设为窗口像素坐标，并使用 `glPointSize(1.0)` 保证一个图元对应一个屏幕像素。
3. **DDA 浮点累计误差的可视化。**当直线较长（如跨越窗口）时，DDA 与 Bresenham 的输出在末端有 1 ~ 2 像素的偏差。

解决：这并非 bug，而是浮点误差累积的预期表现。通过将 DDA 与 Bresenham 平移 10 像素并行绘制，可直观对比两者末端偏差，进一步说明 Bresenham 在长线段上的优越性。

4. 端点丢失问题。初版循环条件写为 `while(x < x2)`，终点像素 (x_2, y_2) 未被绘制。

解决：改为 `for(int i=0; i<=dx; ++i)` 或在循环结束后显式补绘端点。

5 程序运行结果或数据

程序窗口大小 600×600 ，分别使用三种算法绘制端点 $P_1(50, 80)$ 、 $P_2(550, 420)$ 的同一条直线段，并将 DDA、中点、Bresenham 的输出分别在 y 方向偏移 $-10, 0, +10$ 像素以便对比。颜色约定：

- 红色 —— Bresenham 算法
- 绿色 —— 中点画线算法
- 蓝色 —— DDA 算法

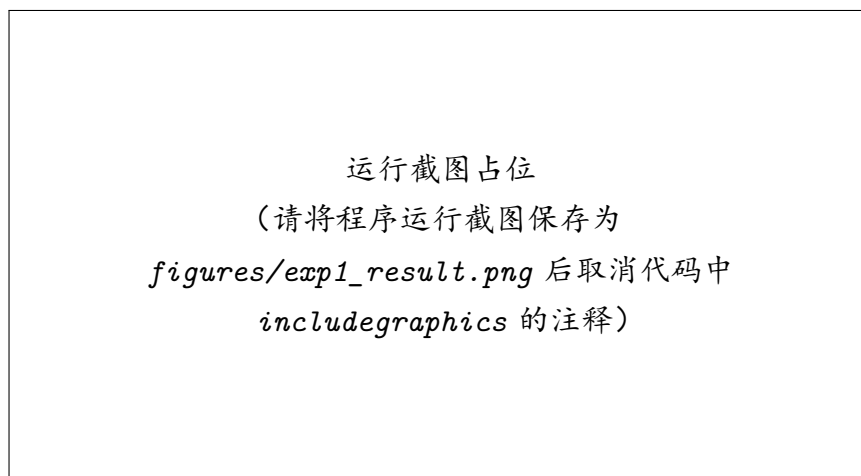


图 1: 三种直线生成算法的对比效果

观察结论：

- 三条直线的整体走向完全一致，验证算法实现正确；
- 在斜率较缓 ($k \approx 0.68$) 的情形下，中点与 Bresenham 输出的像素序列完全相同——这是理论上可证明的等价关系；
- DDA 在直线两端附近的像素分布与另两者偶有 1 像素差异，符合浮点取整误差的预期。

6 实验收获及体会

通过本次实验，我深入体会到了“光栅化”在计算机图形学中的核心地位——再优雅的几何模型，最终都要经过扫描转换才能呈现在屏幕上。三种算法的演进过程（DDA → 中点 → Bresenham）也是一次典型的算法优化历程：从浮点到整数、从乘法到加法、从绝对计算到增量递推，每一步都体现了“用代数等价变形换取硬件友好性”的工程思想。

特别是 Bresenham 算法，仅用整数加减就能精确逼近真实直线，这种巧思在嵌入式 GPU、图形硬件流水线中至今仍被广泛采用。此外，本次实验也让我熟悉了 OpenGL 的基本框架：显示回调 → 投影矩阵 → 图元绘制 → 缓冲区交换，为后续圆弧、曲线、填充等实验打下了基础。

7 参考源程序

完整 OpenGL/C++ 源程序如下，按 1/2/3 键切换显示算法，按 0 键同时显示三种算法。

```
1  // =====
2  // 实验一：直线的生成 —— DDA / 中点 / Bresenham
3  // 编译: cl /EHsc exp1.cpp freeglut.lib opengl32.lib glu32.lib
4  // =====
5  #include <GL/freeglut.h>
6  #include <cmath>
7  #include <cstdio>
8  #include <algorithm>
9
10 const int W = 600, H = 600;
11 int mode = 0;          // 0:全部 1:DDA 2:中点 3:Bresenham
12 int px1 = 50, py1 = 80, px2 = 550, py2 = 420;
13
14 // ----- 工具：单像素绘制 -----
15 inline void putPixel(int x, int y) {
16     glBegin(GL_POINTS);
17     glVertex2i(x, y);
18     glEnd();
19 }
20
21 // ----- 1. DDA 算法 -----
22 void DDA_Line(int x1, int y1, int x2, int y2) {
23     int dx = x2 - x1, dy = y2 - y1;
```

```
24     int steps = std::max(std::abs(dx), std::abs(dy));
25     float xInc = (float)dx / steps;
26     float yInc = (float)dy / steps;
27     float x = (float)x1, y = (float)y1;
28     for (int i = 0; i <= steps; ++i) {
29         putPixel((int)std::lround(x), (int)std::lround(y));
30         x += xInc; y += yInc;
31     }
32 }
33
34 // ----- 2. 中点画线算法 (0 <= k <= 1) -----
35 void Midpoint_Line(int x1, int y1, int x2, int y2) {
36     int a = y1 - y2, b = x2 - x1;
37     int d = 2 * a + b;
38     int d1 = 2 * a;
39     int d2 = 2 * (a + b);
40     int x = x1, y = y1;
41     putPixel(x, y);
42     while (x < x2) {
43         if (d < 0) { ++x; ++y; d += d2; }
44         else      { ++x;      d += d1; }
45         putPixel(x, y);
46     }
47 }
48
49 // ----- 3. Bresenham 算法 (任意斜率) -----
50 void Bresenham_Line(int x1, int y1, int x2, int y2) {
51     int dx = std::abs(x2 - x1), dy = std::abs(y2 - y1);
52     int sx = (x1 < x2) ? 1 : -1;
53     int sy = (y1 < y2) ? 1 : -1;
54     int err = dx - dy;
55     int x = x1, y = y1;
56     while (true) {
57         putPixel(x, y);
58         if (x == x2 && y == y2) break;
59         int e2 = 2 * err;
60         if (e2 > -dy) { err -= dy; x += sx; }
61         if (e2 < dx) { err += dx; y += sy; }
62     }
63 }
```

```
64
65 // ----- 显示回调 -----
66 void display() {
67     glClear(GL_COLOR_BUFFER_BIT);
68     glPointSize(1.0f);
69
70     // 蓝色: DDA ( y - 10 )
71     if (mode == 0 || mode == 1) {
72         glColor3f(0.0f, 0.4f, 1.0f);
73         DDA_Line(px1, py1 - 10, px2, py2 - 10);
74     }
75     // 绿色: 中点 ( 原位 )
76     if (mode == 0 || mode == 2) {
77         glColor3f(0.0f, 0.7f, 0.0f);
78         Midpoint_Line(px1, py1, px2, py2);
79     }
80     // 红色: Bresenham ( y + 10 )
81     if (mode == 0 || mode == 3) {
82         glColor3f(1.0f, 0.0f, 0.0f);
83         Bresenham_Line(px1, py1 + 10, px2, py2 + 10);
84     }
85     glFlush();
86 }
87
88 // ----- 键盘回调 -----
89 void keyboard(unsigned char key, int, int) {
90     if (key >= '0' && key <= '3') {
91         mode = key - '0';
92         glutPostRedisplay();
93     }
94     if (key == 27) std::exit(0); // ESC 退出
95 }
96
97 // ----- 初始化 -----
98 void init() {
99     glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
100     glColor3f(0.0f, 0.0f, 0.0f);
101     glMatrixMode(GL_PROJECTION);
102     gluOrtho2D(0.0, (GLdouble)W, 0.0, (GLdouble)H);
103 }
```

```
104
105 int main(int argc, char** argv) {
106     glutInit(&argc, argv);
107     glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
108     glutInitWindowSize(W, H);
109     glutInitWindowPosition(100, 100);
110     glutCreateWindow("Exp.1 Line Generation: DDA / Midpoint / Bresenham");
111     init();
112     glutDisplayFunc(display);
113     glutKeyboardFunc(keyboard);
114     glutMainLoop();
115     return 0;
116 }
```

Listing 1: 直线生成算法对比——完整源代码